OPENLIST

Collaborative Task & Notes Manager

Flutter MVP — Full Feature Documentation

v2.0 | February 2026

Stack	Timeline	Model
Flutter + Supabase	**10 Days (MVP)**	**B2B SaaS · $7/user**

1. Problem Statement

Teams constantly juggle at least three separate tools: a document editor for notes, a task manager for assignments, and a messaging app for real-time coordination. The friction of switching between these tools causes context loss, stale information, and coordination failures — especially in remote or hybrid settings.

Core Problem

No single lightweight tool lets a small team create a shared note, convert any line of it into an assigned task with sub-items, upload supporting images, and see every change from teammates in real time — all while working smoothly offline.

2. Target Audience

Remote-first teams of 2–15 people: startups, freelance squads, and small dev teams who share a workspace and need real-time task coordination without paying for enterprise tools. Secondary users are solo power users who want a unified notes-plus-tasks editor that syncs across their devices.

3. Proposed Solution — OpenList

A local-first Flutter app that merges collaborative notetaking with real-time task management inside shared Spaces. Every piece of content — text, heading, checklist item, image — is an Atomic Block that can be converted, reordered, or assigned without losing its history.

Core Concepts

- Shared Spaces — a collaborative workspace joined by multiple users, each with a role (Admin or Member). Notes and task lists live inside Spaces.
- Atomic Blocks — every row inside a note or task is a typed reactive object: text, heading, checklist, image, or sub-task.
- Pinned Items — tasks and notes can be pinned to the dashboard per-user or per-Space for quick access.
- Real-time Push — any block added or modified by one user is broadcast via Supabase Realtime to all active members within sub-100ms.
- Task Completion Flow — completing a task updates the shared progress ring and removes it from everyone's pinned view. An in-app notification is sent to all Space members.
- Admin Reminders — when an Admin removes or reassigns a task, all affected members receive an in-app and push notification and update happens on their end too.

4. Full Feature List

Authentication & Onboarding

- Email/password sign-up and login via Supabase Auth (JWT)
- User profile: display name, avatar, plan tier visible in sidebar footer
- Persistent session using flutter_secure_storage

Dashboard

The main landing screen after login, modelled on the HTML reference layout.

- Progress ring / bar — shows active, completed, and overdue task counts for the current user's Spaces
- Pinned Notes strip — vertically scrollable cards of pinned notes, unpinned automatically when a note's tasks are fully completed
- Today's Tasks — filtered task list for tasks due today across all joined Spaces
- Quick-Add input bar — inline task creation with tag chips, due date picker, assign-to selector, and reminder toggle (mirrors the HTML reference)
- Team avatar cluster — shows active collaborators in the current view, clickable to see their profile
- Notification bell — badge count, opens notification panel

Sidebar Navigation

Persistent on tablet/desktop; drawer on mobile. Matches the HTML reference structure.

- Inbox — all unassigned or unread items across Spaces
- Today / Upcoming — date-filtered views
- Notes — flat list of all notes the user has access to
- Lists section — colour-coded Space list (Personal, Work, Team, etc.)
- Bottom panel — user avatar, display name, plan badge, Settings icon

Content Types Inside Notes & Tasks

Every item in a Note or Task supports mixed rich content via Atomic Blocks:

- Text block — plain paragraph with inline bold, italic, underline
- Heading block — H1 / H2 section titles inside the document
- Checklist block — interactive checkbox row, toggleable; acts as a sub-task when nested under a Task
- Image block — upload an image (JPEG / PNG / WEBP ≤ 10 MB) stored in Supabase Storage; displayed inline
- Bullet list block — unordered list item
- Sub-task block — a child task row that links to its own Detail Section page

Task Detail Page

Tapping a task row from the Dashboard or any list navigates to the Task Detail page.

- Full block editor — all content types above are editable inline
- Sub-task list — collapsed by default; each sub-task can be expanded to its own Section page
- Assignee picker — add/remove Space members as assignees
- Due date and reminder configuration
- Activity log — timestamped list of changes (who edited, added, or completed a block)
- Share / copy link (V2)

Sub-task Section Page

Clicking a checklist or sub-task row deep-links into its own full-page view — identical to the Task Detail layout but scoped to that sub-item. Supports its own blocks, assignees, and due date.

Sharing & Multi-User Access

- Space creation — any user can create a Space and invite others by email or link
- Roles — Admin (can delete/reassign any item, manage members) and Member (can create/edit own items, view all)
- Shared Notes — a Note created inside a Space is visible and editable by all members; changes sync in real time
- Shared Lists — a Task List inside a Space shows all tasks regardless of creator; assignee filters available
- Delete Space / Remove Member (Admin only)

Real-Time Collaboration

- Any block add, edit, delete, or reorder triggers a Supabase Realtime event broadcast to all Space members
- Optimistic local update first (Isar) — the sender sees the change immediately, others see it within sub-100ms
- Online presence indicator — avatar highlighted green in team cluster if the user is active in the same document

Notifications & Reminders

- In-app notification panel — bell icon with unread badge; lists recent events
- Task completed — all Space members notified: '{User} completed {Task}'. Removed from everyone's pinned view.
- Task deleted by Admin — all assignees notified: '{Admin} removed {Task} from {Space}'
- Task assigned to you — push + in-app notification
- Task due reminder — local scheduled notification (flutter_local_notifications) at user-set time
- Image uploaded to shared note — in-app event for all members

5. User Flow

The navigation hierarchy follows a strict depth path to keep orientation clear.

1	Login / Sign-up → JWT issued, user profile loaded from Supabase
2	Dashboard → Progress bar, pinned notes strip, today's task list, Quick-Add bar, sidebar
3	Sidebar tap (Space / List) → filtered task/note list for that Space
4	Task row tap → Task Detail Page (full block editor, assignees, sub-tasks, activity log)
5	Sub-task / checklist tap → Section Detail Page (scoped block editor for that sub-item)
6	Note card tap → Note Detail Page (same block editor, shareable within Space)
7	Task checked complete → Progress bar updates, item removed from pinned, Space members notified
8	Admin deletes task → All assignees receive in-app + push notification

9	Any member edits a shared block → All online members see the delta via Realtime in <100ms

6. Tech Stack

Frontend	Flutter (Dart)
State Management	Riverpod + riverpod_annotation + build_runner
Navigation	go_router (deep linking for task/section detail pages)
Local Database	Isar (NoSQL — flexible schema for Atomic Block documents)
Cloud Backend	Supabase (PostgreSQL + RLS + Realtime + Storage + Auth)
Authentication	Supabase Auth — JWT, stored via flutter_secure_storage
Real-time Sync	Supabase Realtime WebSockets — delta broadcast to Space members
Image Storage	Supabase Storage (buckets per Space, RLS-gated)
Push Notifications	Firebase Cloud Messaging (FCM) for remote; flutter_local_notifications for local reminders
Distribution	Firebase App Distribution (Android) / TestFlight (iOS)

7. Data Models

The following tables represent the Supabase (PostgreSQL) schema. Isar mirrors the same structure locally for offline-first operation, with a sync_status field ('synced' | 'pending' | 'conflict') on each record.

users

id	uuid (PK, from Supabase Auth)
display_name	text
avatar_url	text (nullable, Supabase Storage path)
plan	enum: free | pro
fcm_token	text (nullable — for push notifications)
created_at	timestamptz

spaces

id	uuid (PK)
name	text
color_hex	text (sidebar dot colour)

created_by	uuid (FK → users.id)
created_at	timestamptz

space_members

space_id	uuid (FK → spaces.id)
user_id	uuid (FK → users.id)
role	enum: admin | member
joined_at	timestamptz

items (Tasks, Notes, Lists — unified)

id	uuid (PK)
space_id	uuid (FK → spaces.id, nullable for personal items)
parent_id	uuid (FK → items.id, nullable — for sub-tasks/sections)
type	enum: task | note | list | section
title	text
is_pinned	boolean (per dashboard)
is_completed	boolean
due_date	timestamptz (nullable)
reminder_at	timestamptz (nullable)
created_by	uuid (FK → users.id)
updated_at	timestamptz (used for LWW conflict resolution)
order_index	integer (for drag-reorder)

item_blocks (Atomic Block rows inside an item)

id	uuid (PK)
item_id	uuid (FK → items.id)
type	enum: text | heading | checklist | image | bullet | sub_task
content	text (plain text, markdown, or Supabase Storage URL for images)
is_checked	boolean (for checklist/sub_task blocks)
order_index	integer
updated_at	timestamptz

item_assignees

item_id	uuid (FK → items.id)

user_id	uuid (FK → users.id)
assigned_by	uuid (FK → users.id)
assigned_at	timestamptz

notifications

id	uuid (PK)
user_id	uuid (FK → users.id — recipient)
type	enum: task_completed | task_deleted | task_assigned | item_added | reminder
item_id	uuid (FK → items.id, nullable)
actor_id	uuid (FK → users.id — who triggered it)
message	text
is_read	boolean (default false)
created_at	timestamptz

8. System Architecture

Clean layered architecture in Flutter. The UI never touches the network. All layers communicate downward only.

Layer Overview

Presentation	Flutter widgets + Riverpod providers + go_router. Read-only from Isar streams. All writes go through use-cases.
Domain	Pure Dart use-case classes. No Flutter, no Supabase, no Isar imports. Business rules live here.
Data	Two repository implementations per entity: IsarRepository (local) and SupabaseRepository (remote). Injected via Riverpod.
SyncManager	A Dart Isolate running a delta queue. Reads 'pending' records from Isar, upserts them to Supabase, marks them 'synced'. Never blocks the main thread.
Realtime Layer	A Supabase Realtime subscription per joined Space. On receiving a broadcast, it writes the delta directly to Isar and notifies Riverpod providers to rebuild.

Conflict Resolution

MVP strategy: Last Write Wins via updated_at timestamp. If a remote delta arrives with a newer updated_at than the local Isar record, the remote wins and Isar is overwritten. This is a documented tradeoff — field-level CRDT merging is deferred to V2.

Image Upload Flow

- User picks image via image_picker plugin
- Compressed locally to ≤ 1 MB using flutter_image_compress
- Uploaded to Supabase Storage bucket: spaces/{space_id}/images/{uuid}.jpg
- Public URL (or signed URL for private spaces) written as the block.content of an image block
- Realtime broadcast notifies other Space members; they load the URL from their Isar replica

Notification Architecture

- In-app: A Supabase Realtime channel on the notifications table (filtered by user_id) delivers new rows to the client instantly. Bell badge increments; panel lists unread items.
- Remote push: A Supabase Database Webhook (or Edge Function) fires on notification INSERT. It calls FCM v1 API with the recipient's fcm_token to deliver a push notification when the app is backgrounded.
- Local reminders: flutter_local_notifications schedules a local alarm at reminder_at time stored in Isar. No server dependency — works fully offline.

Authentication & Security

- Supabase Auth issues JWTs on login, refreshed automatically by supabase_flutter client
- Row Level Security (RLS) on all Supabase tables — users can only read/write records belonging to Spaces they are members of
- Supabase Storage bucket policies enforce that only space_members can read/write images in their Space's bucket
- flutter_secure_storage holds the refresh token; never in shared preferences or plain files

Deployment Strategy

Mobile (Android)	Firebase App Distribution — internal track for QA builds
Mobile (iOS)	TestFlight (requires Apple Developer account, $99/yr — optional for MVP)
Database & Auth	Supabase Cloud managed instance (free tier for MVP, $25/mo Pro on launch)
Storage	Supabase Storage (included in Pro plan, 100 GB)
Push Notifications	Firebase project (free tier sufficient for MVP user counts)
V2 Backend	Dockerized Node.js on Railway — for advanced conflict resolution, webhooks, Slack/GitHub integrations

> **Architecture Decision — No Custom Backend for MVP**
> All access control is handled by Supabase RLS policies. Push notifications are triggered by Supabase Edge Functions or Database Webhooks calling FCM directly. A custom Node.js/TypeScript server is a V2 addition when webhook orchestration, complex team permissions, and third-party integrations (Slack, Jira) are required.

9. Offline Working & Sync Strategy

OpenList treats the local device as the primary source of truth at all times. Every write — whether online or offline — goes to Isar first. The UI never waits for the network. A background SyncManager isolate queues pending changes and flushes them to Supabase the moment connectivity is restored, automatically broadcasting updates to all Space members in real time.

Each Isar record carries a sync_status field

synced — confirmed on Supabase **pending** — offline, queued **conflict** — remote version is newer; resolved by LWW (updated_at)

Reconnect Flow (Pull-First)

- SyncManager pulls all Space records modified after the device's last synced_at timestamp and writes any remote-newer records to Isar first.
- Local pending deltas are then upserted to Supabase in created_at order with original updated_at timestamps preserved.
- Supabase Realtime broadcasts confirmed changes to all active Space members — no manual refresh required on any device.

What Works Fully Offline

Action	Offline Behaviour
Create / edit any task, note, or block	Saved to Isar instantly; synced on reconnect
Check a task complete	Marked locally; progress ring updates; members notified on reconnect
Upload an image	Cached locally; upload queued; block shows local path until synced
View all tasks and notes	Reads from Isar — fully available, zero network dependency
Local reminders	Fire via flutter_local_notifications — no server needed
Real-time teammate updates / Space invites	Requires connectivity. Realtime subscription resumes and catches up automatically on reconnect.

Known Limitation (V2)

LWW resolves at the record level. If two users edit different blocks in the same note while both offline, one user's changes may overwrite the other's on reconnect. Block-level CRDT merging is deferred to V2.

10. AI-Powered Note-to-Plan Conversion

Users paste raw meeting notes into any Note and tap "Convert to Plan". OpenList sends the text to Google Gemini 2.5 Flash, which extracts tasks, infers due dates, suggests assignees, assigns priorities, and groups items by theme — all returned as structured JSON. The result appears on a Plan Preview screen where the user reviews and edits everything before confirming. Only on confirmation are tasks written to Isar as real Atomic Blocks and broadcast to Space members via Supabase Realtime in the usual <100ms window. The original note is never modified.

AI Stack

Component	Detail
Model	Google Gemini 2.5 Flash — fast, low-cost, 1M token context. Free tier for MVP; paid tier required before production launch.
Dart Package	google_generative_ai (official SDK). Prompt forces JSON-only output, parsed via dart:convert into List<ExtractedTask>.
Architecture	GeminiRepository (Data layer) → ConvertNoteToTasksUseCase (Domain layer). UI never touches the API.
Monetization	Pro plan only. Free users see a paywall sheet on tap. Enforced via the existing plan field feature flag.

Privacy & Consent

Notes are never sent automatically
Extraction is always manually triggered. No note content is sent during auto-save, background sync, or Realtime broadcast. A one-time consent modal informs the user before the first call: "The text of this note will be sent to Google Gemini. Remove passwords, PINs, or confidential data before continuing." The free tier of Gemini may use inputs for model training — switching to the paid tier before any external user accesses this feature is a documented pre-launch deployment gate.

Performance & Real-Time Sync

The AI call (~2–4 seconds) and the Supabase Realtime pipeline are completely separate systems on separate threads. The <100ms collaborative sync guarantee is unaffected. The API latency is a one-time, user-initiated wait shown with a loading indicator — it does not touch any collaborative flow.

Suggested assignees from the AI are free-text strings only — never auto-matched to Space members. The user always makes the final assignment from the real member list in the review screen. If JSON parsing fails, the user sees a toast: "Couldn't extract tasks — try simplifying your notes."

Known Limitations

Limitation	Detail & Mitigation
Extraction Accuracy	Model may miss nuanced context or misclassify priority. Mitigated by the mandatory Plan Preview — no task is saved without user approval.
Sensitive Data	Notes containing passwords or personal data are included in the API call if not removed first. Consent modal and button tooltip warn the user explicitly.
Free Tier Privacy	Google's free tier may use inputs for training. MVP internal use only. Paid tier required before any external user can access this feature.
Assignee Matching	AI suggests names from note text but cannot query Space membership. User manually maps suggestions to real members in the review screen. V2 may add fuzzy matching.

11. Monetization Model

Free	1 Space, 3 active items, personal use only, 7-day sync history, no push notifications
Pro	$10 / user / month — unlimited Spaces, real-time collaboration, image uploads, push notifications, full history
Team	$8 / user / month (billed annually) — everything in Pro + admin controls, usage analytics, priority support

Monetization gates enforced via the plan field on the users table, checked by Supabase RLS rules and Flutter-side feature flags. No paywall complexity in MVP — all features unlocked for internal testing.

12. Scaling Strategy (Post-MVP)

- Supabase read replicas for teams distributed across regions
- PostgreSQL row-level partitioning per space_id for large workspace isolation
- Stateless Node.js sync/webhook layer (V2) scaled horizontally via Railway or AWS App Runner
- CDN-fronted Supabase Storage for low-latency global image delivery
- Rate limiting on block writes per Space (Supabase Edge Functions middleware) to prevent abuse in free tier

13. MVP Timeline (10 Days)

Day	Phase	Focus	Gate
1–2	**Foundation**	Clean Architecture folders. Isar schemas for Item, Block, User, Space, Notification. Riverpod providers. go_router routes (Dashboard → TaskDetail → SectionDetail). Supabase project init + RLS policies.	App navigates between empty screens
3–4	**Local Editor**	Atomic Block widget (text / heading / checklist / image). Inline type-switching. Local CRUD via Isar. 60 FPS validated with 50-block document. Quick-Add bar. Dashboard progress ring.	Full offline editor, no network
5–6	**Auth + Spaces**	Supabase Auth sign-up/login. Space creation + invite by email. space_members RLS. Sidebar Space list with colour dots. User profile in sidebar footer. flutter_secure_storage for JWT.	Two users can log in and join a Space
7–8	**Realtime Sync**	SyncManager isolate (delta queue → Supabase upsert). Realtime subscription per Space. Image upload to Supabase Storage + inline display.	Two devices sync edits in <100ms

		Conflict resolution: LWW via updated_at.	
9	**Notifications**	notifications table + Realtime channel per user_id. Bell badge + notification panel UI. Local reminders via flutter_local_notifications. Admin delete/complete triggers notification row insert.	Notification panel shows live events
10	**Polish & Ship**	Bug fixes. Edge cases (offline→reconnect queue flush, image load errors). Firebase App Distribution build. Recorded 3-minute demo walkthrough video.	Shippable internal APK + demo video

14. Expected Outcome

By the end of Day 10, OpenList will demonstrate:

- ✓ Offline-first block editor with sub-10ms local interactions
- ✓ Multi-user shared Spaces with Admin/Member role enforcement
- ✓ Rich content: text, headings, checklists, images inline in any note or task
- ✓ Three-level navigation: Dashboard → Task Detail → Sub-task Section
- ✓ Real-time delta sync via Supabase Realtime WebSockets (<100ms)
- ✓ Notification system: in-app panel + local reminders; Admin action broadcasts
- ✓ Progress tracking: dashboard ring updates on completion, unpins completed items for all users
- ✓ Distributable internal build + demo suitable for investor or team review

15. Known Tradeoffs & Deferred Decisions

Conflict Resolution	Last Write Wins via updated_at for MVP. CRDT / Operational Transform for field-level merging is V2.
No Custom Backend	Supabase RLS + Edge Functions cover MVP security. Node.js middleware added in V2 for webhook orchestration and third-party integrations (Slack, GitHub, Jira).
Push Notifications (iOS)	FCM push on iOS requires Apple Push Notification service (APNs) certificate setup. Android-first for MVP is acceptable if Apple account is unavailable.
Isar Maintenance Risk	Isar is high-performance but has had community maintenance concerns. Migration path to Drift exists — the Repository Pattern abstracts local storage so swapping the implementation requires only the data layer.
Image Size Limit	10 MB cap per image in MVP. Supabase Storage quota and compression pipeline reviewed before Pro launch.

Presence Indicators	Avatar green-ring for online presence is MVP-lite (Supabase Presence channel). Accurate collaborative cursor positions (like Notion/Figma) are V2.